交付发布、运维与可观测性

从测试通过到安全上线，从发现故障到系统性改进

开发完成不等于上线成功，架构师必须把发布、回滚、观测和复盘提前纳入设计。

核心问题

系统上线后如何知道它健康？

本讲产出

M4：测试计划 V1 + 发布运维方案 V1

课程连接

从质量门禁进入发布和运行

课程链

从测试计划到发布运维方案

测试让变更可验证，发布让变更可控，可观测性让运行状态可解释。

上一讲
质量门禁、测试金字塔、M4 测试计划

→

本讲
CI/CD、发布策略、可观测性、复盘

→

下一讲
AI 时代的软件工程收束

承接
质量门禁只有嵌入流水线，才会变成硬约束。

本讲动作
课堂提问：测试通过以后，还缺什么？

M4 里程碑：把系统带到上线状态

M4 不是再写一份文档，而是证明系统具备安全发布和故障定位的基本能力。

测试计划 V1

覆盖测试层级、质量门禁和关键场景

发布运维方案 V1

发布策略、回滚、监控指标、观测接入点

评分重点

量化阈值、理由、可执行步骤

评分锚点：回滚触发条件必须量化。

上线第一天：三个真实感场景

真正危险的不是出问题，而是问题发生后不知道发生在哪里。

1. 通知消失

审批通知始终不到，不知道消息是否发出

2. 接口变慢

查询从 200ms 变成 3s，没有错误只有慢

3. 需要回滚

故障设备被标记为可预约，必须快速退回

共同问题：上线后是否能发现、定位、恢复？

三场景背后的共同缺失

缺的不是工具名，而是可重复发布、可量化回滚和可观测运行。

缺的不是工具名，而是可重复发布、可量化回滚和可观测运行。

发布不可重复
只能靠某个人手工操作

回滚不可量化
不知道何时回、回到哪、多久完成

运行不可解释
没有日志、指标、追踪形成证据链

复盘不可落地
只说下次注意，没有系统改进

名词锚点：可重复、可审计、可回溯。

手工发布的三个缺陷

手工发布最大的问题不是慢，而是不可重复、不可审计、不可回溯。

不可重复

同一套步骤换个人可能不一样

→

不可审计

事后不知道谁在何时改了什么

互动：你们项目现在怎么发布？

CI/CD 的核心价值

CI/CD 把发布从个人手艺变成团队可执行、可追踪的工程流程。

CI

每次提交后自动构建、测试、扫描

CD

通过门禁后可部署到预发布或生产

质量门禁

测试和扫描不通过，流水线自动停止

版本制品

每次发布都有可追踪版本标签

CI/CD 流水线：从提交到验证

流水线的关键是把构建、测试、制品、部署和验证串成可审计链路。

质量门禁

预约系统每次发布都应能追踪到 commit、镜像标签和测试报告。

考试锚点

考试：指出流水线中的质量门禁。

CI/CD

质量门禁嵌入流水线才算硬约束

门禁不是会议上说“要重视质量”，而是流水线中无法绕过的自动检查。

单元测试
快速发现局部逻辑错误

集成测试
验证数据库、接口和外部依赖

质量扫描
代码规范、安全漏洞、复杂度

冒烟测试
部署后快速确认主流程可用

流水线门禁：失败即阻断发布

GitHub Actions 骨架：看三件事就够

配置文件不需要逐行背，先看触发条件、质量门禁和版本制品。

```yaml
name: CI/CD Pipeline
on: push
jobs:
  build-and-test:
    steps:
      - checkout
      - mvn clean package
      - mvn test
      - mvn verify -P integration-test
      - mvn sonar:sonar
      - docker build -t reservation:${{ github.sha }}
  deploy-staging:
    needs: build-and-test
    steps:
      - ./deploy.sh staging ${{ github.sha }}
      - ./smoke-test.sh staging
```

触发

push 到 main/develop 自动运行

测试

mvn test + verify + sonar

制品

Docker 镜像携带 commit 标签

部署

develop 部署到 Staging 并冒烟

制品、环境与版本标签

能回滚的前提，是每次发布都有明确制品和环境边界。

制品不可变

同一镜像标签不应被反复覆盖

环境分层

开发、测试、预发布、生产边界清晰

版本标签

commit SHA / release tag 可追踪

发布记录

谁、何时、发布了什么、结果如何

发布策略是架构决策

发布策略决定故障影响范围、回滚速度和数据兼容性，不只是运维操作。

发布策略决定故障影响范围、回滚速度和数据兼容性，不只是运维操作。

影响范围
错误版本影响多少用户

回滚速度
能否分钟级恢复

数据兼容
新旧版本是否共享数据库

观测能力
能否判断新版本是否健康

简答：为什么发布策略属于架构决策？

发布策略

蓝绿部署：双环境与快速切换

蓝绿部署用一套旧环境兜底，让回滚从重新部署变成流量切换。

负载均衡器
流量入口

蓝环境（旧版本）

发布前承载 100% 流量；发布后保留作回滚预案

绿环境（新版本）

先部署验证；发布后 100% 流量切到绿

适用场景：快速回滚优先。

蓝绿最容易忽视数据库

蓝绿切流很快，但数据库通常只有一套，新旧版本必须兼容。

只增加不删除

先加字段，旧版本仍可运行

双写/兼容读

迁移期避免新旧字段互相破坏

旧版本退出

确认无旧流量后再清理

回滚验证

切回旧版本后数据仍可用

发布策略

灰度发布：先让少数用户试水

灰度发布用小流量暴露风险，但前提是有足够用户量和监控能力。

负载均衡器
按比例分流

旧版本
90% 流量，稳定兜底

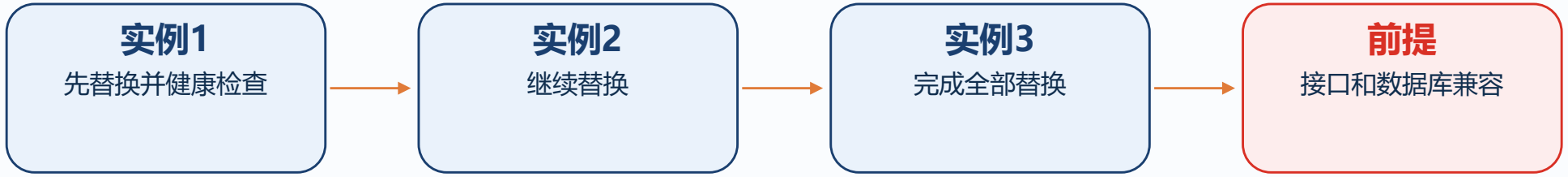
金丝雀新版本
10% 流量，观察错误率和延迟

放量路径：5% → 20% → 50% → 100%

发布策略

滚动更新：逐实例替换

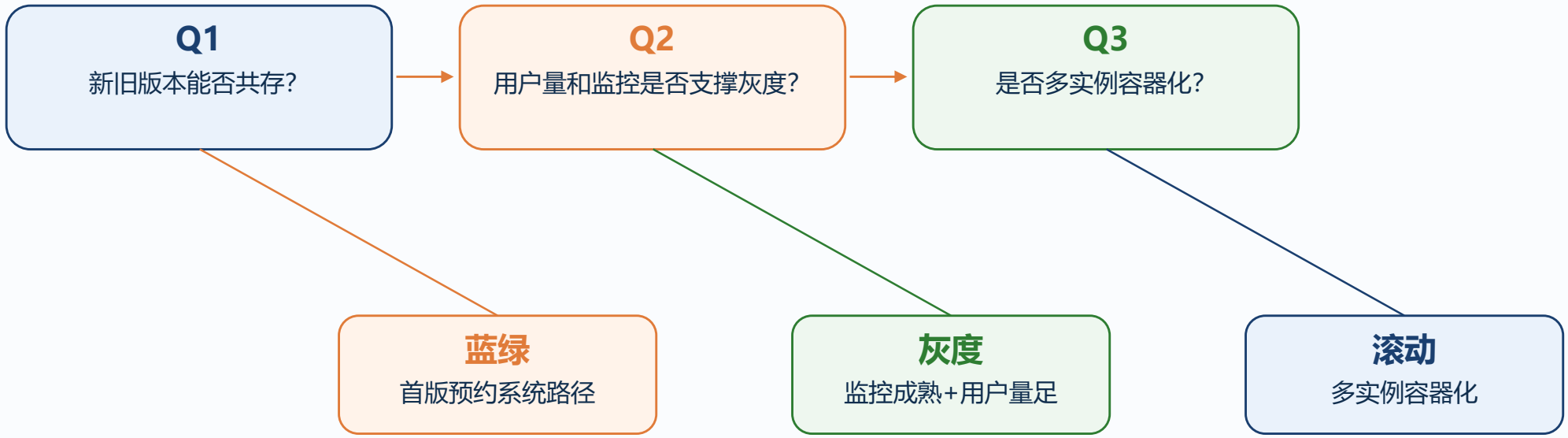
滚动更新节省资源，但要求多副本部署且新旧版本短暂共存。



适用：多副本容器化。

发布策略决策树：三问定方向

选择发布策略时，先问能否共存，再问监控能力，最后问部署形态。



预约系统首版路径：共存需谨慎、监控较弱、单机房 → 蓝绿。

预约系统首版：为什么选蓝绿

首版选择蓝绿，不是因为它最先进，而是因为它最符合团队和风险约束。

团队

5人，发布流程应简单

用户

约 2000 人，不适合精细灰度统计

回滚

故障设备可预约必须快速恢复

资源

可承受短时双环境

回滚方案四问

没有触发条件、目标版本、数据策略和耗时估计的回滚方案，不可执行。

何时回滚

错误率、失败率、人工判定阈值

回到哪里

版本标签 / 蓝环境 / 上一稳定镜像

数据怎么办

扩展迁移、补偿、兼容读写

多久完成

预计切换和验证耗时

回滚触发条件要量化

> “感觉不对就回滚”不可执行，阈值必须事先写进方案。

预约创建错误率	> 5% 持续 5 分钟	立即回滚
通知投递失败率	> 20% 持续 10 分钟	降级并评估
P95 响应时间	> 2s 持续 5 分钟	停止扩大流量
故障设备可预约	出现 1 例	立即回滚

M4：至少写 3 条量化阈值。

监控 vs 可观测性

监控发现已知问题，可观测性帮助理解未知问题。

监控

预设阈值：错误率高、CPU 高、接口慢

→

可观测性

面对未知问题，能追查发生了什么、在哪里、为什么

名词解释：监控与可观测性。

只有一个支柱会怎样

日志、指标和追踪各有盲区，三者组合才构成运行理解能力。

只有日志

信息太多，定位慢，难聚合

只有指标

知道慢了，但不知道慢在哪里

只有追踪

看单次请求，不知道整体规模

整体健康 + 具体链路 + 事件证据

结构化日志：让日志能被机器理解

可检索、可聚合、可关联的日志，才是故障定位材料。

非结构化日志

```
WARN 预约失败
user=1024 device=示波器
通知异常，请稍后查看
```

结构化 JSON 日志

```
{
  "timestamp": "2026-05-10T10:20:31Z",
  "level": "ERROR",
  "service": "reservation-api",
  "traceId": "abc123",
  "spanId": "s2",
  "userId": "1024",
  "deviceId": "OSC-01",
  "action": "createReservation",
  "duration_ms": 3200
}
```

代码题：指出 JSON 日志关键字段。

预约系统结构化日志字段

日志字段设计要服务定位问题，而不是为了“多记一点”。

timestamp
何时发生

userId/deviceId
哪个用户、哪台设备

traceId/spanId
属于哪次请求和哪个步骤

duration_ms/error
耗时和错误证据

审批通知没发时，traceId 能把预约、审批、通知链路串起来。

关键指标：系统健康的仪表盘

指标要同时覆盖基础设施和业务，否则只能看到一半事实。

CPU/内存	基础设施	>80% 持续 10 分钟
DB连接池	基础设施	可用连接 < 20%
HTTP错误率	接口	>5% 持续 5 分钟
预约成功率	业务	<95% 持续 10 分钟
通知送达率	业务	<90% 持续 10 分钟

M4：至少 5 条指标，含业务和基础设施。

预约系统业务指标不能缺

业务指标直接回答用户是否真的完成了目标。

预约成功率

核心流程是否可用

审批平均耗时

流程是否堵塞

通知送达率

异步链路是否健康

冲突失败数

强一致规则是否生效

链路追踪：看一次请求走过哪里

追踪把分散调用组织成一棵 Span 树，定位慢点和断点。

Trace
一次完整请求

Span
请求中的一个步骤

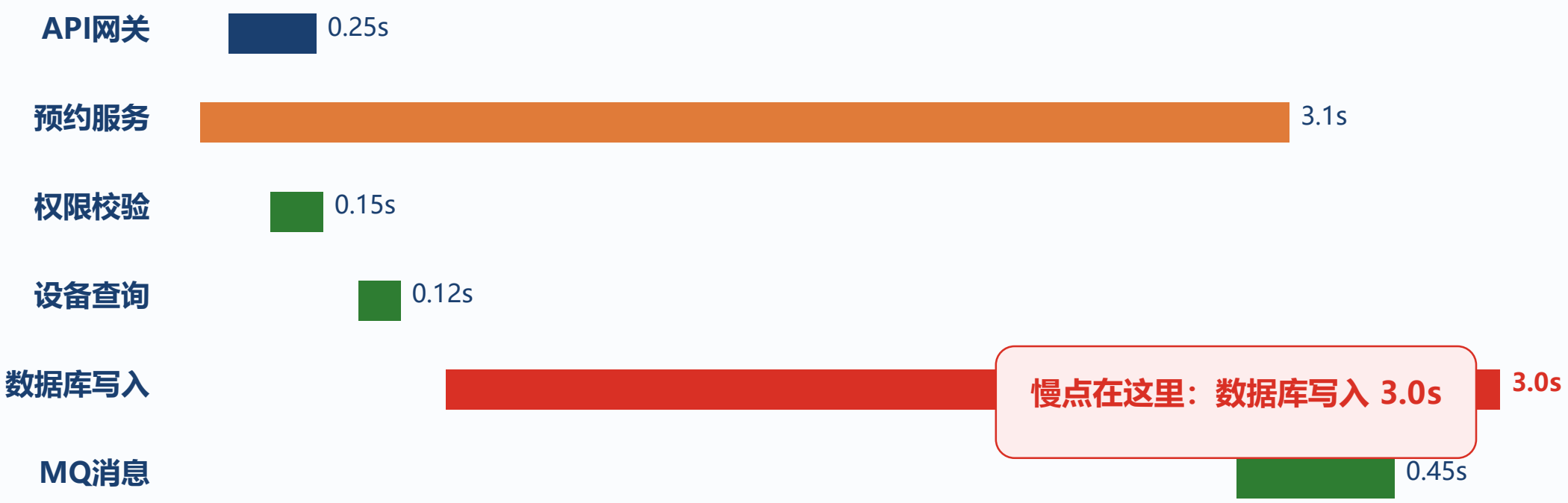
上下文传递
跨服务保留 traceId

瀑布图
用耗时条显示慢点

可观测性

链路追踪瀑布图：3.2秒慢在哪里

追踪的价值是把“接口慢”定位到具体 Span。



互动：看到瀑布图后先查哪里?

OpenTelemetry：可观测性的标准化入口

OTel 统一埋点 API 和数据格式，降低被具体监控后端锁定的风险。

OTel 统一埋点 API 和数据格式，降低被具体监控后端锁定的风险。

统一 SDK
应用代码按标准埋点

统一协议
日志、指标、追踪可统一导出

后端可切换
Prometheus、Jaeger、Tempo 等

架构价值
观测接入点在设计阶段预留

快速记忆：OTel 是标准，不是一个仪表盘。

预约系统最小可观测性方案

最小方案先覆盖核心业务事件、关键流程节点和跨服务边界。

预约创建

记录 userId、deviceId、reservationDate

审批动作

记录 approver、approvalResult

通知发送

记录 channel、deliveryStatus

TraceID传递

异步消息也要传递上下文

复盘不是追责会议

Post-mortem 的目标是减少下次故障概率，不是找一个人背锅。

追责式

X 同学操作失误，下次注意

→

系统式

为什么单人操作能造成故障，哪里缺校验、审批、回滚和监控？

判断：哪种复盘能防止下次发生？

无效复盘 vs 有效复盘

有效复盘必须给出系统性根因和可验证改进项。

无效结论

下次更认真、加强意识

系统根因

流程、架构、监控、自动化缺陷

可落地改进

Owner + Deadline + 验证方式

反馈闭环

改进项进入下一轮架构和流程

故障复盘

5 Whys：从表象追到系统性根因

连续追问为什么，是为了离开表面故障，找到架构和流程缺口。

表象
用户收不到审批通知

→

Why1
通知服务崩溃

→

Why2
队列积压导致 OOM

→

Why3
通知服务无弹性扩容

→

Why4
未考虑高峰场景

→

Why5
设计阶段缺容量规划

第5个为什么通常才接近可改进的系统性原因。

故障复盘

预约系统 5 Whys 演练

复盘要带着学生一层层追问，而不是直接展示答案。

表象
大量用户投诉收不到审批通知

→

根因
容量规划缺失，未识别早上开课前高峰

→

改进1
新增高峰时段识别

→

改进2
队列深度告警 >1000

→

改进3
通知服务 HPA 最大 5 副本

通知服务 OOM 只是表象，不是终点。

故障复盘

时间线还原：发现时间就是损失时间

复盘要精确到分钟，尤其要看问题发生到被发现之间的间隔。

10:00	通知队列开始积压	无人知晓
10:20	用户开始投诉	客服/管理员
10:45	工程师发现队列异常	开发团队
11:05	通知服务重启并清理队列	全团队

M4：监控指标要缩短发现时间。

Post-mortem 标准结构

标准模板保证团队讨论事实、根因和改进，而不是情绪和责任。

摘要
影响、时间、范围

根因分析
系统性原因和触发因素

时间线
事件、谁知道、行动

改进行动
Owner、Deadline、状态、验证方式

改进项要可验证

改进项如果不能验证，就只是口号。

不可用写法

加强监控、提高意识、规范流程

可用写法

队列深度 >1000 告警，Owner 张三，5月20日前上线

验证方式

压测触发队列积压，确认告警到达

状态跟踪

Open / Doing / Done

演练

Cloudflare 2019 BGP 路由泄露事故

大型事故往往不是单点错误，而是外部触发因素与系统防线不足叠加。

背景

小型 ISP 发布错误 BGP 路由

影响

Cloudflare 网络拥塞，约 90 分钟影响

关键点

错误路由被接受，过载保护不足

复盘价值

看系统性根因和架构改进

演练

Cloudflare 时间线还原

时间线能把“事故发生”与“团队知道”之间的差距显性化。

11:02	错误 BGP 路由发布，流量异常进入	无人知晓
11:10	工程师收到告警，开始排查	工程师
11:42	识别根本原因，开始处置	团队
12:00	恢复 90% 服务	全团队

互动：如果发现提前到 1 分钟，损失会怎样?

Cloudflare 根因与改进

好的复盘把触发因素、系统性根因和架构改进区分开。

触发因素

外部 ISP 发布错误路由

系统性根因

路由验证、防护和监控响应不足

架构改进

推进 RPKI、改进监控、调整过载保护

可迁移启发

外部依赖异常也要有防线

演练

课堂演练：30分钟事故复盘

演练目标是产出时间线、5 Whys 和三条架构改进建议。

分组
4-6 人一组，10 分钟阅读材料

任务1
还原时间线和发现延迟

任务2
区分触发因素和系统性根因

任务3
提出 3 条可验证改进项

课堂动作：阅读材料 10 分钟 + 讨论 10 分钟 + 汇报 10 分钟

演练讲评：复盘质量怎么看

复盘质量取决于能否从事故故事推导出可执行的系统改进。

事实清楚

时间线精确，影响范围明确

根因清楚

不把外部触发当最终根因

改进清楚

Owner、Deadline、验证方式完整

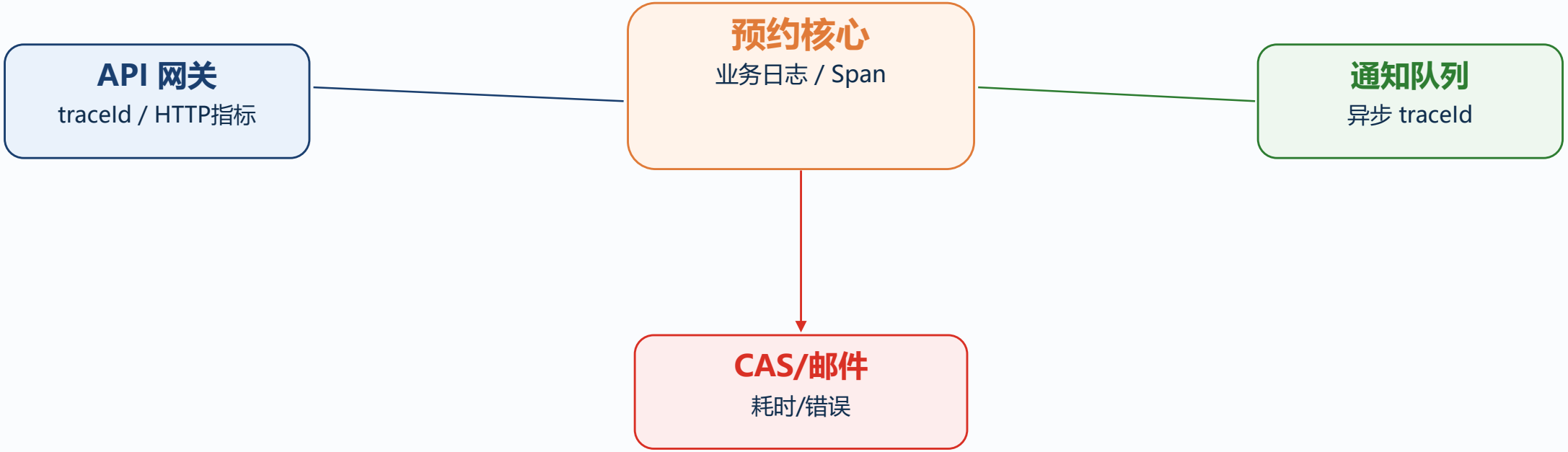
迁移清楚

能回到本组项目改进 M4

收束

把可观测性标回架构图

可观测性不是上线后补日志，而是架构图上可见的接入点。



M4：架构图上标注观测点。

M4 发布运维方案 V1 模板

M4 要交一份能执行的上线方案，而不是 DevOps 名词清单。

1. 发布策略

策略选择、流程、频率、理由

2. 回滚方案

触发阈值、步骤、DB策略、耗时

3. 监控指标

至少5条，含业务和基础设施

4. 观测接入点

TraceID、关键日志、指标采集

收束

M4 评分重点：可执行、可验证

方案写得越像现场操作手册，评分越高。

发布策略	是否援引项目约束
回滚条件	是否量化阈值
指标清单	是否业务+基础设施都有
接入点	是否标明 TraceID/日志/指标位置
复盘改进	是否有 Owner 和 Deadline

M4 自查清单。

AI 可以辅助交付，但不能替代审查

AI 适合生成流水线骨架和脚本草案，但生产权限、密钥和回滚策略必须人工审查。

适合AI

CI配置骨架、脚本模板、日志字段建议

必须人工

secrets、权限边界、生产审批

风险

生成脚本可能误用明文密钥或过宽权限

原则

AI 生成，工程师负责

安全

发布流水线里的安全边界

流水线拥有部署权，一旦密钥和权限失控，风险会直接进入生产环境。

Secrets

不得写入仓库和日志

最小权限

流水线只拿必要部署权限

审批

生产部署可增加人工确认

审计

记录谁触发、部署了什么版本

本讲六条关键判断

上线能力是架构能力的一部分，发布、观测、复盘要形成闭环。

1. 开发完成不等于上线成功
2. CI/CD 让发布可重复、可审计、可回溯
3. 发布策略决定影响范围和回滚速度
4. 可观测性三支柱互补
5. 复盘要找系统性根因
6. M4 必须可执行、可验证

考试与答辩锚点

本讲考核重点是判断和应用，不是工具名背诵。

名词解释

CI/CD、蓝绿、灰度、可观测性、5 Whys

简答题

三种发布策略适用条件

案例题

给故障场景设计日志/指标/追踪

大作业

M4 发布运维方案可执行性

结束

从运行闭环走向 AI 时代工程能力

下一讲将系统讨论 AI 工具介入后，软件工程哪些变了，哪些必须坚持。

下一讲将系统讨论 AI 工具介入后，软件工程哪些变了，哪些必须坚持。

课后任务

完成 M4 发布运维方案 V1

下讲预告

AI 时代的软件工程与架构判断

最终目标

让项目从能运行，走向能交付、能观察、能演进

问题交流：M4 有哪些不清楚?